AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 1–2

แก้ให้ผลสแกนถูกต้อง และทำให้รันทั้งระบบได้ด้วยคำสั่งเดียว
พร้อมเหตุผลว่าแต่ละจุดแก้เพราะอะไร

> โปรเจกต์: `C:\Allprojects\Projects`
> ทดสอบกับ: NodeGoat (Semgrep, Trivy fs, Trivy image, ZAP)
> วันที่: 6 ตุลาคม 2026

1. สรุปสั้นๆ

ก่อนแก้ ระบบเป็นสคริปต์ทดลองหลายไฟล์ที่ต้องรันเองทีละไฟล์ ผูกกับ NodeGoat แบบตายตัว
และมีหลายจุดที่ทำให้**ผลในรายงานผิดโดยไม่มีใครรู้** เช่น ZAP อ่านผลสแกนเก่า หรือช่องโหว่ที่แก้แล้วยังขึ้นในรายงาน

หลังแก้ รันทั้งระบบได้ด้วยคำสั่งเดียว ใช้กับโปรเจกต์อื่นได้ และถ้าเครื่องมือตัวไหนพัง
ระบบจะบันทึกและแจ้งให้รู้แทนที่จะเงียบหรือใช้ข้อมูลผิด

เรื่อง	ก่อนแก้	หลังแก้
วิธีรัน	รันเอง 5–6 สคริปต์ตามลำดับ	`python run_pipeline.py ...` คำสั่งเดียว
ใช้กับโปรเจกต์อื่น	ต้องแก้ path ชื่อโปรเจกต์ และ URL ในโค้ด	ส่งเป็น argument ได้เลย
เครื่องมือสแกนพัง	crash ทั้งระบบ หรือแย่กว่านั้นคือใช้ผลเก่าแบบเงียบๆ	บันทึกเป็น `failed` พร้อม error แล้วสแกนตัวอื่นต่อ
สถานะการสแกน	ค้างที่ `running` ตลอด	`completed` / `partial` / `failed` พร้อมเวลา
ช่องโหว่ที่แก้แล้ว	ยังเป็น `open` และขึ้นในรายงานตลอด	เปลี่ยนเป็น `fixed` อัตโนมัติ รายงานแสดงเฉพาะ `open`
การนับช่องโหว่ซ้ำ	บางจุดรวมกันจนหาย บางจุดแตกเป็นหลายรายการ	fingerprint แม่นขึ้น (Semgrep จาก 22 เป็น 35 จุดจริง)
รายงาน	เขียนทับไฟล์เดิม วันที่ตายตัว	แยกโฟลเดอร์ทุกรอบ วันที่จริง บันทึกลงตาราง `reports`

วิธีใช้ตอนนี้

ผลลัพธ์อยู่ที่ `reports\<ชื่อโปรเจกต์>_<วันที่_เวลา>\` ประกอบด้วย `dashboard.html`, `report.html` และ `security_report.pdf`

```
.\venv\Scripts\Activate.ps1
python run_pipeline.py --project NodeGoat `
    --repo C:\Allprojects\Targets\NodeGoat `
    --image nodegoat:latest `
    --url http://host.docker.internal:4000
```

Argument	ความหมาย
`--project` (บังคับ)	ชื่อโปรเจกต์ ใช้แยกข้อมูลในฐานข้อมูล
`--repo`	โฟลเดอร์ source code ใส่แล้วจะรัน SAST (Semgrep) และ Dependency (Trivy fs)
`--image`	ชื่อ Docker image ใส่แล้วจะรัน Container scan (Trivy image)
`--url`	URL ของเว็บที่รันอยู่ ใส่แล้วจะรัน DAST (ZAP) ถ้าเว็บรันบนเครื่องตัวเองให้ใช้ `host.docker.internal` แทน `localhost` เพราะ ZAP รันใน Docker
`--scans`	เลือกเองว่าจะรันอะไร เช่น `--scans sast,dast`
`--no-report` / `--output-dir`	สแกนอย่างเดียวไม่สร้างรายงาน / เปลี่ยนโฟลเดอร์เก็บรายงาน (ค่าเริ่มต้น `reports`)

2. สิ่งที่แก้: ให้ผลสแกนถูกต้อง

กลุ่มนี้สำคัญที่สุด เพราะถ้าไม่แก้ **รายงานจะแสดงข้อมูลผิดโดยที่ดูไม่ออก** ซึ่งอันตรายกว่าระบบที่พังให้เห็นตรงๆ

1 ตรวจว่าเครื่องมือสแกนทำงานสำเร็จจริง สำคัญมาก

ปัญหาเดิม

โค้ดเรียก `semgrep` / `trivy` แล้วนำ stdout ไปแปลงเป็น JSON ทันที โดยไม่ดูว่าโปรแกรมจบแบบสำเร็จหรือ error

ทำไมเป็นปัญหา

ถ้าไม่ได้ติดตั้งโปรแกรม เน็ตหลุด หรือ image ไม่มีอยู่จริง stdout จะว่าง แล้วเจอ `JSONDecodeError` ที่ไม่บอกเลยว่าพังเพราะอะไร และ crash ทั้งระบบ ทำให้สแกนตัวอื่นที่ยังไม่ได้รันหายไปด้วย

แก้อย่างไร

สร้างไฟล์ `scanners/runner.py` เป็นตัวกลางที่ทุก scanner ใช้ร่วมกัน ทำ 3 อย่าง:

- ตรวจ **exit code** โดยรู้ว่าเครื่องมือแต่ละตัวใช้ค่าไหนแปลว่าสำเร็จ (ZAP คืน 1–2 เมื่อเจอช่องโหว่ ซึ่งไม่ใช่ error)
- ตั้ง **timeout** (30 นาที) กันโปรแกรมค้าง
- แปลง error เป็นข้อความที่อ่านเข้าใจ เช่น "ไม่พบโปรแกรม 'semgrep' กรุณาติดตั้ง..." แล้วโยนเป็น `ScannerError`

ไฟล์: `scanners/runner.py` (ใหม่), `scanners/sast.py`, `scanners/dependency.py`, `scanners/container.py`

2 ZAP ไม่อ่านผลสแกนรอบเก่าแล้ว สำคัญมาก

ปัญหาเดิม

ZAP เขียนผลลงไฟล์ `zap_results.json` ในโฟลเดอร์ NodeGoat แล้วโค้ดก็เปิดไฟล์นั้นมาอ่านตรงๆ

ทำไมเป็นปัญหา

ถ้ารอบนี้ ZAP รันไม่สำเร็จ (เช่น เว็บเป้าหมายไม่ได้เปิด) ไฟล์จากรอบก่อนยังค้างอยู่ **โค้ดจะอ่านผลเก่ามาใช้โดยไม่มีอะไรเตือน** รายงานเลยดูปกติทั้งที่ไม่ได้สแกนจริง แถมยังเขียนไฟล์ขยะลงใน source code ของเป้าหมายด้วย

แก้อย่างไร

สร้าง**โฟลเดอร์ชั่วคราวใหม่ทุกครั้ง**ด้วย `tempfile.mkdtemp()` ทำให้ไม่มีทางเจอไฟล์เก่า ถ้า ZAP ไม่ได้สร้างไฟล์ผลก็แจ้ง error ชัดเจน แล้วลบโฟลเดอร์ทิ้งเมื่อเสร็จ ตอนนี้ไม่ต้องส่ง `work_dir` แล้ว

ไฟล์: `scanners/dast.py`

3 บันทึกสถานะการสแกนให้ถูกต้อง และสแกนต่อแม้มีตัวพัง

ปัญหาเดิม

ScanRun และ ScanJob ถูกสร้างเป็น `"running"` แล้วไม่เคยถูกเปลี่ยนเลย ส่วน `started_at`, `finished_at` และ `error_message` ว่างตลอด

ทำไมเป็นปัญหา

ดูจากฐานข้อมูลแล้วแยกไม่ออกว่าสแกนไหนสำเร็จ ไหนพัง หรือไหนยังรันอยู่ จึงตรวจย้อนหลังไม่ได้ และในอนาคตถ้าทำเป็น API ก็บอกผู้ใช้ไม่ได้ว่างานเสร็จหรือยัง

แก้อย่างไร

เขียนฟังก์ชัน `run_single_scan()` ที่ครอบการสแกนแต่ละตัวด้วย `try/except`:

• สำเร็จ: `completed` พร้อม `finished_at`

• พัง: `failed` พร้อมเก็บ `error_message` **แล้วไปสแกนตัวถัดไปต่อ**

ส่วน ScanJob สรุปจากทุกตัว: ผ่านหมดเป็น `completed`, ผ่านบางตัวเป็น `partial`, พังหมดเป็น `failed`

ไฟล์: `save_findings.py`

4 ช่องโหว่ที่แก้แล้วเปลี่ยนเป็น `fixed` อัตโนมัติ

ปัญหาเดิม

ระบบรู้จักแค่ "เจอใหม่" (insert) กับ "เจอซ้ำ" (update) ถ้านักพัฒนาแก้ช่องโหว่ไปแล้ว finding นั้นยังเป็น `open` ตลอด และรายงานก็ไม่ได้กรองสถานะ

ทำไมเป็นปัญหา

รายงานมีแต่จะยาวขึ้นเรื่อยๆ นักพัฒนาแก้แล้วตัวเลขไม่ลด ไม่มีใครเชื่อรายงาน และวัดความคืบหน้าไม่ได้

แก้อย่างไร

ระหว่างสแกน เก็บ fingerprint ที่เจอไว้ใน set พอสแกนเสร็จ ฟังก์ชัน `mark_missing_as_fixed()` จะหา finding ของเครื่องมือเดียวกันที่ยัง `open` แต่รอบนี้ไม่เจอ แล้วตั้งเป็น `fixed` และรายงานกรองเฉพาะ `status == "open"`

> **จุดสำคัญ:** จะปิด finding **เฉพาะเมื่อการสแกนนั้นสำเร็จ** ถ้า Semgrep พัง ระบบจะไม่ปิด finding ของ Semgrep เพราะ "ไม่เจอ" ในกรณีนั้นแปลว่า "ไม่ได้ดู" ไม่ใช่ "แก้แล้ว" ผมทดสอบกรณีนี้จริงแล้ว

ไฟล์: `save_findings.py`, `generate_report.py`

5 Fingerprint แม่นขึ้น (ตัวระบุว่าช่องโหว่ไหนคืออันเดียวกัน)

Fingerprint คือค่า hash ที่ใช้ตัดสินว่า "finding รอบนี้คืออันเดียวกับรอบก่อนไหม" ถ้าละเอียดไป ของเดียวกันจะแตกเป็นหลายรายการ ถ้าหยาบไป ของต่างกันจะถูกรวมจนหายไป

เครื่องมือ	เดิม	ใหม่	เหตุผล
Semgrep	rule + ไฟล์	rule + ไฟล์ (path สัมพัทธ์) + โค้ดที่ match	เดิมถ้าไฟล์เดียวมีจุดผิดกฎเดียวกัน 3 บรรทัด จะเหลือ 1 รายการ **(ช่องโหว่หายไปจากรายงาน)** ใช้โค้ดแทนเลขบรรทัดเพื่อให้ยังจับคู่ได้แม้บรรทัดเลื่อน และใช้ path สัมพัทธ์เพื่อให้ย้ายโฟลเดอร์ได้
Trivy (dep / container)	package + CVE	package + **เวอร์ชัน** + CVE	npm ติดตั้ง package เดียวกันหลายเวอร์ชันพร้อมกันได้ ซึ่งแต่ละเวอร์ชันต้องแก้แยกกัน
ZAP	plugin + URL เต็ม	plugin + method + URL (ตัด query) + parameter	URL เดียวกันที่ query ต่างกัน (`?id=1` , `?id=2`) เดิมถูกนับเป็นคนละรายการ ทำให้รายงานบวม

ไฟล์: `normalizer.py` | ผลจริง: Semgrep เจอ 35 จุด (เดิมรายงาน 22) และ ZAP เหลือ 91 รายการ (เดิม 155 ที่ซ้ำกันเยอะ)

6 คำนวณ severity และ risk score ตั้งแต่ตอนบันทึก

ปัญหาเดิม

ตอนบันทึก finding ใหม่ ตั้ง `severity_normalized="medium"` ตายตัวทุกตัว และ `risk_score` ว่าง ต้องรอรัน `apply_risk_scoring.py` แยกถึงจะถูก

ทำไมเป็นปัญหา

ถ้าลืมรันขั้นนั้น ทุกช่องโหว่จะเป็น medium หมด ช่องโหว่ critical จะดูเหมือนเรื่องเล็ก

แก้อย่างไร

เรียก `normalize_severity()` และ `calculate_risk_score()` ทันทีใน `save_finding_with_dedup()` ทั้งตอนเพิ่มใหม่และตอนอัปเดต ส่วน `apply_risk_scoring.py` เก็บไว้ใช้เฉพาะตอนเปลี่ยนสูตรแล้วอยากคำนวณข้อมูลเก่าใหม่

ไฟล์: `save_findings.py` , `apply_risk_scoring.py`

7 `requested_scans` ตรงกับการสแกนที่รันจริง

ปัญหาเดิม

บันทึก `requested_scans=["sast"]` แต่จริงๆ รันครบ 4 แบบ

ทำไมเป็นปัญหา

ข้อมูลในฐานข้อมูลไม่ตรงความจริง ถ้าดูย้อนหลังจะเข้าใจผิด

แก้อย่างไร

บันทึกรายการสแกนตามที่ผู้ใช้เลือกหรือตามที่ระบบเลือกจาก argument

3. สิ่งที่แก้: ใช้กับโปรเจกต์อื่นได้ และรันได้ด้วยคำสั่งเดียว

8 ไฟล์ใหม่ `run_pipeline.py`: คำสั่งเดียวรันทั้งระบบ

ปัญหาเดิม

ชื่อ `"NodeGoat"`, path `C:\Allprojects\Targets\NodeGoat`, image `nodegoat:latest` และ URL เขียนไว้ในโค้ดตรงๆ อยากสแกนโปรเจกต์อื่นต้องเข้าไปแก้โค้ด แล้วยังต้องรันหลายสคริปต์ให้ถูกลำดับเอง

แก้อย่างไร

สร้าง `run_pipeline.py` ที่รับค่าผ่าน argument (ดูตารางหน้า 2) แล้วทำตามลำดับ **สแกน → จับคู่ CWE/OWASP → ออกรายงาน**

มีการตรวจ argument ก่อนเริ่ม เช่น เลือก dast แต่ไม่ใส่ `--url` หรือโฟลเดอร์ `--repo` ไม่มีอยู่จริง ก็จะแจ้งทันที ไม่ต้องรอสแกนไปครึ่งทางแล้วค่อยพัง

ถ้าการสแกนทุกตัวล้มเหลว โปรแกรมจะจบด้วย exit code 1 เพื่อให้ใช้ใน CI/CD ได้

9 แยกโค้ดเป็นฟังก์ชัน ไม่รันเองตอน import

ปัญหาเดิม

ทุกไฟล์เขียนโค้ดไว้ระดับบนสุด ไม่มี `def main()` หรือ `if __name__ == "__main__"` ทำให้โค้ดรันทันทีที่ถูก import

ทำไมเป็นปัญหา

ตัวอย่างจริง: `generate_report.py` import `normalize_cwe_id` จาก `apply_cwe_mapping.py` ทำให้**ทุกครั้งที่ออกรายงาน สคริปต์ mapping ทั้งไฟล์จะรันตามไปด้วยโดยไม่ตั้งใจ** และนำโค้ดไปใช้ซ้ำหรือเขียนเทสไม่ได้

แก้อย่างไร

แยกเป็นฟังก์ชัน เช่น `run_scan_job()`, `get_or_create_project()`, `apply_cwe_mapping(session, project_id)`, `apply_risk_scoring(session)`, `build_report_data()` และ `generate_report()` โดยไฟล์เดิมที่ใช้เป็นสคริปต์ยังรันเดี่ยวได้เหมือนเดิมผ่าน `__main__`

ไฟล์: `save_findings.py`, `apply_cwe_mapping.py`, `apply_risk_scoring.py`, `generate_report.py`

10 รายงานแยกโฟลเดอร์ทุกรอบ และบันทึกลงฐานข้อมูล

ปัญหาเดิม

เขียนทับ `security_report.pdf` และ `templates/*.html` ทุกครั้ง วันที่ในรายงานตายตัวเป็น `"2026-09-11"` และมีตาราง `reports` แต่ไม่ได้ใช้

ทำไมเป็นปัญหา

เทียบรายงานรอบก่อนกับรอบนี้ไม่ได้ วันที่ในรายงานผิด และไฟล์ผลลัพธ์ไปปนกับ template

แก้อย่างไร

สร้างโฟลเดอร์ `reports/<โปรเจกต์>_<วันที่_เวลา>/` ใหม่ทุกรอบ ใช้วันที่จริง คัดลอก CSS ไปไว้ด้วยเพื่อให้เปิด HTML ได้ทันที และบันทึกแถวในตาราง `reports` (path ของ PDF และจำนวนตาม severity) ผูกกับ ScanJob นอกจากนี้ `generate_report.py` รันแยกได้ด้วย `--project ชื่อ`

11 จุดเล็กๆ ที่แก้ไปด้วย

จุด	แก้อะไร / ทำไม
`models.py` : `ScanJob.status`	ประกาศ type เป็น `Mapped[int]` แต่จริงๆ เก็บเป็น String แก้ให้ตรง เพื่อไม่ให้ IDE หรือ type checker เข้าใจผิด
`models.py` : UniqueConstraint	มีใน migration แต่ไม่มีในโมเดล เพิ่มให้ตรงกัน ตรวจด้วย `alembic check` แล้วไม่ต้องสร้าง migration ใหม่
ข้อมูลว่างจาก scanner	ใช้ `.get(...) or []` แทน `["Results"]` เพราะ Trivy คืน `null` เมื่อไม่เจอช่องโหว่ ซึ่งเดิมทำให้ crash
fingerprint ซ้ำในรอบเดียวกัน	ZAP อาจรายงาน URL เดียวกันหลายครั้ง ตอนนี้นับครั้งเดียว และ `flush()` ทุกครั้งเพื่อไม่ให้ชน UniqueConstraint
ภาษาไทยใน terminal	ตั้ง stdout เป็น UTF-8 ใน `run_pipeline.py` เพราะถ้า output ถูก pipe หรือเก็บเป็น log บน Windows จะใช้ cp1252 ที่พิมพ์ภาษาไทยไม่ได้
`.gitignore`	เพิ่ม `reports/`, `security_report.pdf`, `zap_results.json` และ `ca-bundle.pem` ซึ่งเป็นผลลัพธ์หรือไฟล์เฉพาะเครื่อง ไม่ควรขึ้น git

4. ปัญหา Norton Antivirus กับ Semgrep

ตอนทดสอบ Semgrep ล้มเหลวด้วย error นี้:

```
SSLError: HTTPSConnectionPool(host='semgrep.dev', port=443) ...
CERTIFICATE_VERIFY_FAILED: unable to get local issuer certificate
```

สาเหตุ

Norton มีฟีเจอร์ **SSL/TLS scanning** ที่ดักเปิดดูข้อมูล HTTPS โดยเอา certificate ของตัวเอง ("Norton Web/Mail Shield Root")
ไปใส่แทนของเว็บจริง Windows และเบราว์เซอร์เชื่อ certificate นี้ แต่ Python ใช้รายการ certificate ของตัวเอง (แพ็กเกจ `certifi`) ซึ่งไม่มีของ Norton จึงปฏิเสธการเชื่อมต่อ

ปัญหานี้**ไม่ได้มาจากโค้ดของเรา** แต่กระทบทุกอย่างที่ใช้ Python เชื่อมต่อ HTTPS บนเครื่องนี้ รวมถึง `pip install` ด้วย

วิธีแก้ที่ทำไปแล้ว

1. รวม certificate ปกติกับของ Norton เป็นไฟล์เดียว:

```
Get-Content venv\Lib\site-packages\certifi\cacert.pem, C:
\ProgramData\Norton\Antivirus\wscert.pem | Set-Content -Encoding ascii ca-bundle.pem
```

2. บอก Python ให้ใช้ไฟล์นี้ โดยเพิ่มใน `.env`:

```
REQUESTS_CA_BUNDLE=C:\Allprojects\Projects\ca-bundle.pem
SSL_CERT_FILE=C:\Allprojects\Projects\ca-bundle.pem
```

โค้ดเรียก `load_dotenv()` ใน `db.py` ซึ่งใส่ค่าเหล่านี้ลงใน environment และ `semgrep` ที่ถูกเรียกผ่าน subprocess ก็ได้รับค่านี้ต่อไปด้วย

> **ผลทดสอบ:** ScanJob 32 สถานะ `completed` และ Semgrep ทำงานปกติ

> **ถ้าวันหลัง Semgrep กลับมาเจอ error นี้อีก** (เช่น หลังอัปเดต Norton หรือรัน `pip install --upgrade certifi`) ให้รันคำสั่งข้อ 1 ใหม่อีกครั้ง ไฟล์ `ca-bundle.pem` ใช้ได้เฉพาะเครื่องนี้ จึงใส่ไว้ใน `.gitignore` แล้ว

5. ผลการทดสอบจริง

รายงานที่ได้: `dashboard.html` แสดง 144 กลุ่มช่องโหว่ (finding ที่ open รวม 336 รายการ) และ `security_report.pdf` สร้างได้ปกติ

ScanJob	ทดสอบอะไร	สถานะ	ผล
28	ใส่ image ที่ไม่มีอยู่จริง	`failed`	ไม่ crash บันทึก error ไว้ใน ScanRun ✔ (ลบข้อมูลทดสอบนี้ออกแล้ว)
29	สแกนครบ 4 แบบกับ NodeGoat	`partial`	Semgrep ล้มเพราะ Norton แต่อีก 3 ตัวรันต่อจนเสร็จ ✔ และ finding เก่าของ Semgrep ไม่ถูกปิดผิดๆ ✔
31	SAST หลังรวม certificate	`completed`	Semgrep เจอ 35 จุด ✔
32	SAST หลังตั้งค่าใน `.env` (รันเอง)	`completed`	Semgrep 35 จุด อัปเดตทั้งหมด ไม่มีรายการซ้ำ ✔ แสดงว่า fingerprint ใหม่จับคู่ได้คงที่

6. สิ่งที่ควรรู้ และงานที่ยังเหลือ

เรื่องที่ต้องตัดสินใจ

> **finding เก่า 358 รายการที่ชี้นเป็น `fixed` ไม่ได้ถูกแก้จริง**
> พอเปลี่ยนวิธีคำนวณ fingerprint (ข้อ 5) finding รอบแรกหลังแก้ได้ค่าใหม่ทั้งหมด ระบบเลยเห็นว่า "ของเดิมหายไป" แล้วตั้งของเดิมเป็น `fixed` ส่วนของใหม่เป็น `open` รายงานไม่ได้รับผลเพราะแสดงเฉพาะ `open` แต่ตัวเลข "แก้ไขแล้ว" ในฐานข้อมูลจะไม่ตรงความจริง **แนะนำให้ลบ 358 รายการนี้ทิ้ง** เพราะเป็นข้อมูลซ้ำ

ข้อจำกัดที่ยังมี

เรื่อง	รายละเอียด
Gemini ยังไม่ได้อยู่ใน pipeline	ถ้าเจอ CWE ใหม่ที่ยังไม่มีวิธีแก้ ต้องรัน `python generate_remediation.py` เอง (ช้า เพราะรอ 13 วินาทีต่อ CWE เพื่อไม่ให้เกินโควตา API) ทำเป็น option `--with-remediation` ได้
สแกนหลาย image ในโปรเจกต์เดียว	การปิด finding ตอนนี้แยกตามเครื่องมือ ถ้าโปรเจกต์เดียวกันรอบนี้สแกน image A รอบหน้า image B ผลของ A จะถูกตั้งเป็น `fixed` ถ้าโปรเจกต์หนึ่งใช้ image เดียวก็ไม่มีปัญหา
`verify_findings.py`	ยังเขียน path `package-lock.json` ของ NodeGoat ไว้ตายตัว
`seed_cwe_mapping.py`	ยังต้องรันเองหนึ่งครั้งตอนตั้งค่าฐานข้อมูลใหม่
ไฟล์เก่าที่ไม่ใช้แล้ว	`security_report.pdf` ที่โฟลเดอร์หลัก, `templates/output_report.html`, `templates/security_dashboard.html` และตาราง backup `cwe_owasp_mapping_backup*` 2 ตาราง

ขั้นต่อไปที่แนะนำ

ขั้น	งาน	ทำไม
3	`requirements.txt`	ให้คนอื่นหรือเครื่องอื่นติดตั้งได้ตรงเวอร์ชัน
3	เทสด้วย pytest	เทส `normalizer`, `risk_scoring` และ `normalize_cwe_id` ด้วย JSON ตัวอย่าง เพื่อให้แก้โค้ดแล้วรู้ทันทีถ้าทำของเดิมพัง (ไฟล์ใน `tests/` ตอนนี้เป็นสคริปต์ลองเรียก ไม่มี `assert`)
3	Commit ขึ้น git	ตอนนี้งานทั้งหมดยังไม่ได้ commit มีแค่ Phase 0 ถ้าไฟล์หายหรือแก้พังจะย้อนกลับไม่ได้
4	API + งานเบื้องหลัง	ทำ endpoint ใน `main.py` สำหรับสั่งสแกน ดูสถานะ และดาวน์โหลดรายงาน แล้วให้สแกนรันเบื้องหลังผ่าน Redis
4	ปรับ risk score	ใช้ EPSS (มีคอลัมน์อยู่แล้วแต่ยังไม่ได้ใช้), confidence และว่ามีเวอร์ชันที่แก้แล้วหรือยัง

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`run_pipeline.py`	`ใหม่` จุดเริ่มรันทั้งระบบ

`scanners/runner.py`	**ใหม่** ตัวกลางเรียกเครื่องมือสแกนและตรวจ error
`scanners/sast.py`, `dependency.py`, `container.py`, `dast.py`	**แก้** ใช้ runner และ ZAP ใช้โฟลเดอร์ชั่วคราว
`save_findings.py`	**เขียนใหม่** เป็นฟังก์ชัน พร้อมสถานะ การปิด finding และ severity
`normalizer.py`	**แก้** fingerprint ของ Semgrep, Trivy และ ZAP
`generate_report.py`	**แก้** เป็นฟังก์ชัน กรอง open แยกโฟลเดอร์ และบันทึกตาราง reports
`apply_cwe_mapping.py`, `apply_risk_scoring.py`	**แก้** เป็นฟังก์ชันและมี `__main__`
`models.py`	**แก้** type ของ status และ UniqueConstraint
`.gitignore`	**แก้** เพิ่มไฟล์ผลลัพธ์และ ca-bundle
`.env`, `ca-bundle.pem`	**คุณทำเอง** ตั้งค่า certificate ของ Norton